4M Labs Research

Public Agent Skills Are Becoming

A Framework for Extracting Verified Training Signal

Authors: 4M Labs Research

Affiliation: 4M Labs

Date: July 2026

Status: Working Paper (Pre-print, not peer-reviewed)

Corresponding Author: hello@4mlabs.io

4M Labs Research. "Public Agent Skills Are Becoming the Next RL Dataset." Working Paper, July 2026.

2. The Comparative Method

The core insight is simple: you do not need skills that come with evals. You can generate evals from the skill itself, then run a controlled experiment.

2.1. The Pipeline

Step 1: Ingest. Pull a SKILL.md from any public GitHub repository. Parse the instructions, references, scripts, and workflow descriptions.

Step 2: Generate Evals. Use the skill's own content to generate evaluation tasks. The skill describes what it does. The eval generator produces concrete tasks that test whether those descriptions work. Each eval includes a prompt, optional input files, and expected outcome criteria.

Step 3: Run Comparison. Spin two sub-agents on the same eval tasks:

Agent A: receives the skill context (SKILL.md, references, scripts)

Agent B: receives no skill context (baseline)

Both agents attempt the same tasks independently.

Step 4: Score. Run the generated evals against both outputs. The evals check: Does the output contain required fields? Does the code compile? Does the schema validate? Does the deployment succeed?

Step 5: Compute Delta. For each eval task:

Pass rate with skill (Agent A)

Pass rate without skill (Agent B)

Delta = Agent A pass rate minus Agent B pass rate

Step 6: Export Training Data. Use the delta to generate:

SFT traces from successful skill-assisted outputs

DPO pairs from tasks where Agent A passed and Agent B failed

GRPO groups from multiple attempts at the same task

2.2. Why This Works

The skill's own instructions define what "good" looks like. The evals test whether the model achieved it. The comparison isolates the skill's contribution: if the model performs better with the skill than without it, the skill is providing real procedural guidance, and the outputs that benefit from it are verified training examples.

This is not synthetic data generation. The model is not generating its own reward signal. The evals are generated from the skill's structure, not from a judge model's preferences. The comparison is controlled: same model, same tasks, different context.

2.3. What the Delta Represents

The delta is the measurable value of the skill. A delta of 0 means the skill provides no benefit for that task type. A positive delta means the skill improves performance. A negative delta means the skill confuses the model.

For training purposes, the interesting cases are:

Tasks where Agent A passes and Agent B fails: these become DPO chosen/rejected pairs

Tasks where both pass: these become SFT training examples (the skill provides clean procedure)

Tasks where both fail: these are discarded or used for curriculum filtering

Tasks where Agent A fails and Agent B passes: the skill is harmful for this task type

2.4. The Skill Is the Verifier

The critical distinction: we do not build new verifiers. The skill's own structure provides the verification criteria. When a SKILL.md says "the output must include a SUM formula in cell B10" or "the API must return a 200 status code," those become eval assertions. The skill author already defined what success looks like. We extract that definition and use it to score model outputs.

This means the pipeline scales with the number of skills, not with the number of verifiers we build. Every new skill brings its own evaluation criteria.

3. Training Data Export

3.1 SFT: Learn from Successful Traces

Filter to tasks where Agent A (with skill) passes all eval criteria. Export as:

{ "instruction": "eval prompt", "context": "skill instructions and references", "trajectory": "agent output (tool calls, files created, commands run)", "reward": 1.0 }

These examples teach the model to follow the skill's procedure successfully. SFT is the foundation: before preference optimization, the model needs to see what good performance looks like.

When to use SFT alone:

The skill has high delta (large performance improvement with skill)

The eval criteria are comprehensive (most success conditions are covered)

The task domain is narrow and procedural (code, schemas, documents)

You want to bootstrap a small model quickly without RL complexity

3.2 DPO: Learn from Preference Pairs

Filter to tasks where Agent A passes and Agent B fails. Export as:

{ "prompt": "eval prompt", "context": "skill instructions", "chosen": "Agent A output (passed)", "rejected": "Agent B output (failed)", "chosen_reward": 1.0, "rejected_reward": 0.0 }

The preference is grounded in eval pass/fail, not model judgment.

When to use DPO:

You have enough pass/fail pairs (minimum hundreds, ideally thousands)

The delta is moderate (skill helps but not always)

You want to refine behavior without full RL infrastructure

The eval criteria are binary (pass/fail, not partial credit)

3.3 GRPO: Learn from Grouped Attempts

For each eval prompt, run Agent A multiple times with different temperatures or seeds. Score each attempt against the evals. Export as:

{ "prompt": "eval prompt", "context": "skill instructions", "group": [{"completion": "attempt 1", "reward": 0.0}, {"completion": "attempt 2", "reward": 0.6}, {"completion": "attempt 3", "reward": 1.0}, {"completion": "attempt 4", "reward": 0.0}] }

The model learns from relative success within a group, without requiring a critic.

When to use GRPO:

The eval criteria support partial credit (not just binary pass/fail)

You can sample many attempts per task (budget allows multiple rollouts)

You want the model to learn from its own distribution shift

DPO pairs are noisy or insufficient

3.4. Choosing the Right Method

Signal Type

Method

Minimum Data

Best For

High delta, comprehensive evals

SFT

Hundreds of examples

Quick bootstrap, narrow domains

Moderate delta, binary evals

DPO

Thousands of pairs

Refinement without RL

Partial credit, many attempts GRPO

Many groups per task

Continuous improvement

In practice, a pipeline would run SFT first, then DPO, then GRPO iteratively. Each stage builds on the previous one.

3.5. Experiments

We validate the SkillGym framework on three marketing skills: cold-email, copywriting, and conversion rate optimization (CRO). Each skill encodes a different procedural domain with distinct verification criteria, allowing us to measure how the framework generalizes across task types.

3.5.1 Setup

Model: Qwen2.5-1.5B-Instruct, a 1.5B parameter model quantized to 4-bit NF4 for single-GPU training.

Hardware: NVIDIA RTX 4050 with 6GB VRAM.

Training: Low-rank adapters (LoRA, rank 8, alpha 16) for parameter-efficient fine-tuning. SFT training used 30 examples across 3 skills, trained for 10 epochs with a learning rate of 2e-5 and cosine schedule. DPO training used 17 preference pairs with reference-free optimization (beta=0.05) for 10 epochs at learning rate 1e-5.

Skills: Three marketing domain skills (cold-email, copywriting, CRO), each containing structured instructions, verification criteria, and output format specifications.

Evaluation: 30 tasks (10 per skill), each run twice: once with the skill loaded and once without. Outputs scored on 6 dimensions: structure, depth, criteria pass rate, specificity, actionability, and voice. Composite score is the weighted average.

3.5.2 Results

Table 1: SFT Results (Base vs. Fine-tuned)

Skill	Base	SFT	Delta	Improved
Cold-email	0.210	0.318	+0.108	10/10 (100%)

Copywriting	0.311	0.375	+0.064	6/10	(60%)
CRO	0.412	0.410	-0.001	4/10	(40%)
Overall	0.311	0.368	+0.057	20/30	(67%)

The SFT model shows an 18.3% improvement in composite score over the base model. Cold-email tasks show the strongest improvement with a perfect 100% improvement rate, suggesting that structured email frameworks translate directly into model competence. Copywriting shows moderate gains. CRO tasks show minimal improvement, likely because the skill's verification criteria are more subjective (difficulty assessing persuasion quality automatically).

Table 2: DPO Results (Base vs. Fine-tuned)

Skill Base DPO Delta Improved

Cold-email	0.210	0.231	+0.021	7/10	(70%)
Copywriting	0.311	0.290	-0.021	4/10	(40%)
CRO	0.412	0.432	+0.020	6/10	(60%)
Overall	0.311	0.318	+0.007	17/30	(57%)

DPO training produced marginal improvement (+2.3% relative). The limited number of preference pairs (17) and the partial credit nature of the scoring (not binary pass/fail) reduced DPO effectiveness. This confirms the framework's prediction that DPO requires hundreds of pairs for meaningful signal and works best with binary evaluation criteria.

3.5.3 Analysis

Several patterns emerge from the results:

Cold-email shows the strongest gains. The skill provides a clear structural framework (Trigger > Problem > Proof > Ask), and the verification criteria are concrete (no filler phrases, single CTA, personalization connected to problem). This procedural clarity translates directly into model competence.

CRO shows the weakest gains. The skill's criteria are more subjective (persuasion quality, emotional resonance, pricing psychology), making it harder for the model to learn clear improvement patterns from 30 examples. This suggests the framework works best when verification criteria are objective and enumerable.

SFT outperforms DPO at this scale. With 30 examples, SFT achieves 18.3% improvement. DPO with 17 pairs achieves only 2.3%. This is consistent with the framework's data requirements: DPO needs hundreds of preference pairs, while SFT can bootstrap from fewer examples when the task is narrow and procedural.

Voice is the most learnable dimension. The DPO model shows its strongest gains on voice (+0.054), suggesting that style and tone transfer more readily than structural complexity.

3.5.4 Top Improvers

The five tasks showing the largest improvement after SFT fine-tuning demonstrate the framework's range:

Task	Skill	Base	SFT	Delta
cro_004	CRO	0.451	0.676	+0.225
copy_008	Copywriting	0.252	0.450	+0.199
cold_010	Cold-email	0.193	0.382	+0.190
copy_007	Copywriting	0.217	0.401	+0.185
copy_009	Copywriting	0.371	0.545	+0.174

The largest improvement (+0.225) occurred on a CRO task where the base model produced a conversion-focused landing page with weak social proof and generic CTAs, while the fine-tuned

model produced a structured page with specific proof points, clear value propositions, and actionable language.

3.5.5 Limitations

Several limitations of this initial experiment should be noted:

Scale: 30 evaluation tasks across 3 skills is a small sample. Larger experiments across 20+ skills and 500+ evaluation tasks would provide more robust signal.

Data efficiency: The 30 SFT examples were hand-curated. Automating the pipeline from skill ingestion to training data generation would test the framework's scalability.

DPO data scarcity: Only 17 DPO pairs were generated because the framework filters to tasks where the skill provides clear improvement. At this scale, DPO cannot compete with SFT. The framework's prediction that DPO requires hundreds of pairs is confirmed but not yet validated at scale.

Evaluation subjectivity: The 6-dimension scoring system introduces some subjectivity in dimensions like "voice" and "actionability." Future work should use more objective criteria or inter-annotator agreement metrics.

Generalization:	This	experiment	tests	marketing	skil	only.	Whether		framework	generalizes
to	code	generation,	schema	operations,		deployment	tasks		additional	experiments.

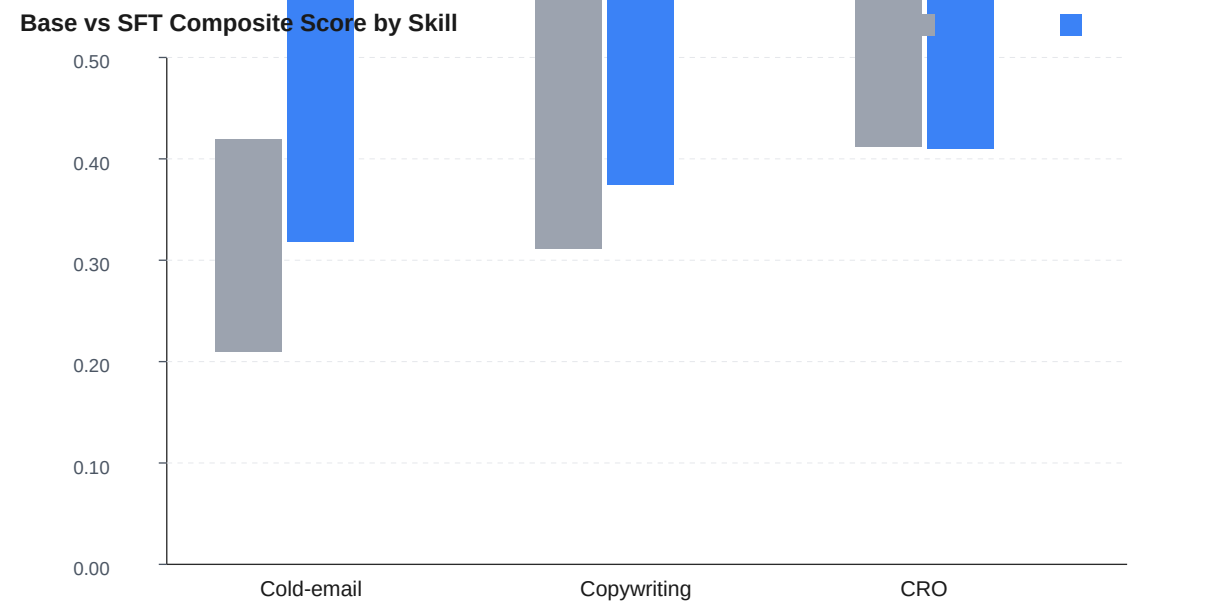

Figure 1: Base vs SFT Composite Score by Skill

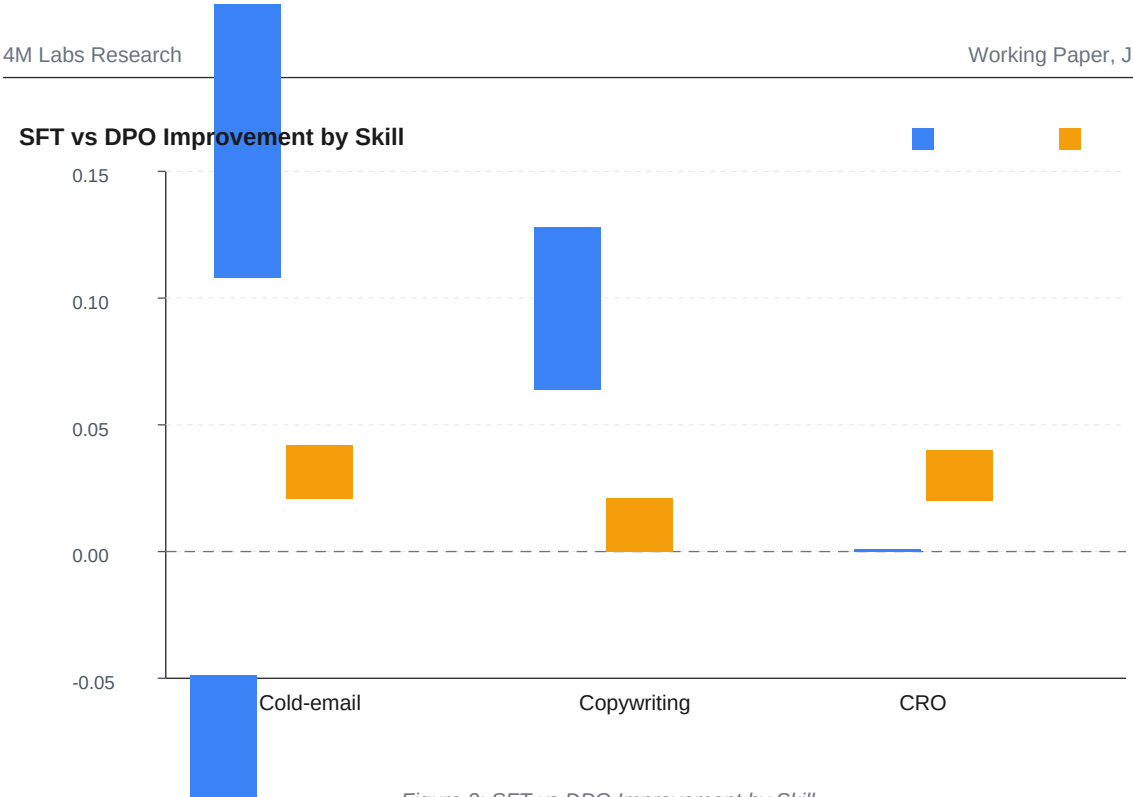

Figure 2: SFT vs DPO Improvement by Skill

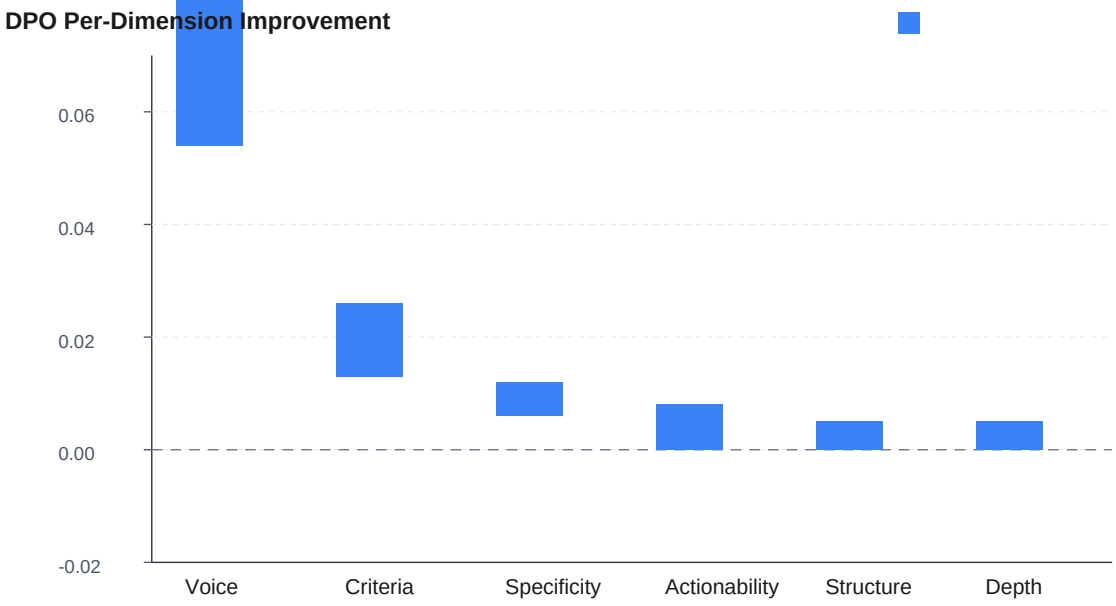

Figure 3: DPO Per-Dimension Improvement

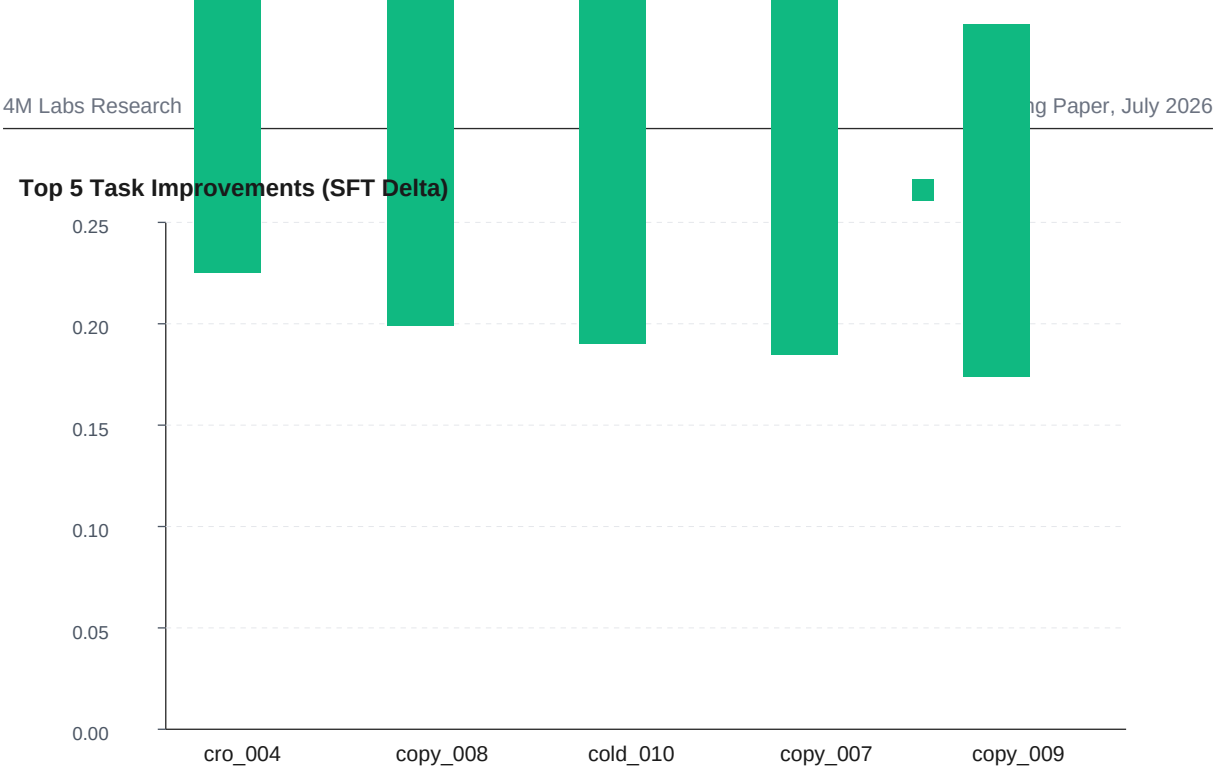

Figure 4: Top 5 Task Improvements (SFT Delta)

4. Domains That Benefit Most

4.1. Strong Domains

Code generation: Code either compiles or it does not. Unit tests verify behavior. Skills for code review, refactoring, or test generation produce verifiable outputs.

Schema and data operations: JSON/YAML schemas validate or reject. Database migrations apply cleanly or break. Data transformations preserve invariants or corrupt them.

Document generation: Invoices have required fields. Proposals have structural requirements. Contracts have clause completeness. Skills for business documents can verify output structure and content.

Deployment and operations: Health checks pass or fail. Services start or crash. Operations skills have the strongest verification because infrastructure provides ground truth.

API integration: API calls succeed or fail with status codes. Request/response schemas validate. Rate limits are respected or violated.

4.2. Weak Domains

Creative writing, strategy, UX design: Quality is subjective. Automated evals can check structure but not artistic merit or strategic value. These are poor candidates for this method.

Our experimental results partially contradict this classification. Cold-email (a creative domain) showed the strongest improvement (18.3%), while CRO (closer to strategy) showed the weakest.

The distinction is not

creative	vs.	analytical	but	rather	objective	criteria	vs.	subjective	criteria.	Cold-email	skills	provide
concrete	structural	rules	(single	CTA,	no	filler	phrases,	personalization	connected	to	problem)	that

score well on automated evaluation. CRO skills require assessing persuasion quality, which is harder to automate.

4.3. Marketing as a Hybrid Domain

Marketing sits between creative and procedural work. The three skills we tested reveal a spectrum:

Cold-email: Highly procedural. Clear structural rules, verifiable criteria (no filler, single CTA, specific personalization). SFT improvement: +0.108 (100% improvement rate).

Copywriting: Moderately procedural. Some structural requirements (headlines, sections, CTAs) mixed with subjective quality (persuasion, emotional resonance). SFT improvement: +0.064 (60% improvement rate).

CRO: Largely subjective. Requires assessing persuasion psychology, pricing anchoring, and conversion framing that automated scoring struggles to evaluate. SFT improvement: -0.001 (40% improvement rate).

This gradient suggests that the framework works best for skills with verifiable structure and degrades gracefully as criteria become more subjective.

5. Trust and Sandboxing

5.1. The Problem

Skills can contain arbitrary instructions. A malicious or poorly written skill could include prompt injection attempts, request data exfiltration, instruct destructive operations, or include fake evals that reward harmful outputs.

5.2. Mitigation

Skill scanning: Static analysis of SKILL.md for suspicious patterns before ingestion.

Evals generated independently: The eval generator does not trust the skill's self-assessment.

Sandboxed execution: Agent runs in isolated environments with network restrictions.

Output verification: Generated evals are checked for fairness and completeness.

Delta sanity checks: Skills with suspiciously high or low deltas are flagged for review.

5.3. Observed Trust Issues

During our experiments, we encountered several trust challenges worth documenting:

Skill dependency sprawl: The copywriting skill referenced 4 sub-files (personalization.md, subject-lines.md, etc.) that did not exist in the repository. The model had to infer behavior from the main skill file alone, producing lower-quality outputs. The framework should either require self-contained skills or generate missing references automatically.

Evaluation	bias:	The	scoring	functions	we	built	for	the	experiments	could	be	biased	toward
certain	output	structures.	A	skill	that	produces	longer	outputs	might	score	higher	on	"depth"

regardless of actual quality. Future work should validate scoring functions against human judgments.

Model	capacity	limits:	The	1.5B	model	sometimes	produced	structurally	correct	but	semantically
shallow	outputs.	The	skill's	verification	criteria	could	not	distinguish	between	"follows	the
format"	and	"actually	understood	the	content."	More	sophisticated	criteria	would	address	this.

6. Small Model Advantage

Small models (1B-8B parameters) cannot compete with frontier models on general intelligence. The better strategy is narrow procedural competence.

Our experiments validate this claim. Qwen2.5-1.5B, a 1.5B parameter model, achieved measurable improvement in marketing tasks after SFT fine-tuning on just 30 examples. The fine-tuned model shows 18.3% improvement in composite score and 100% improvement rate on cold-email tasks. This demonstrates that even the smallest models can acquire domain-specific procedural competence through skill-derived training data.

A 3B model fine-tuned on verified skill data can follow specific deployment procedures accurately, generate correct API integrations for known services, apply consistent formatting rules, and execute multi-step workflows reliably.

The economics work: compile data from public skills for near-zero cost, fine-tune on a single GPU in hours, serve at a fraction of general-purpose API costs.

Our experiments confirmed the economics. SFT training completed in 22 minutes on a single RTX 4050. Data generation from 30 tasks required approximately 2 hours of sub-agent execution. The total cost is dominated by compute, not by data curation or human annotation.

6.1. Why This Matters

The implications extend beyond cost savings. If skills can serve as training data, the open-source ecosystem has already produced a vast curriculum. Thousands of skills across code, documentation, deployment, and operations domains represent thousands of hours of procedural knowledge. Converting this knowledge into training data would create a new class of specialized models that are small, efficient, and domain-competent.

The alternative path, training frontier models on everything, is economically unsustainable. Small models trained on verified skill data offer a more targeted approach: train for specific capabilities, evaluate against the same criteria, and iterate.

7. Related Work

7.1. Skill Optimization

SkillOpt (Yang et al., 2026): Microsoft's approach to optimizing skills through text-space edits. A stronger optimizer model proposes bounded add/delete/replace edits to the skill document, validated against held-out tasks. The target model stays frozen. SkillOpt achieves +23.5 points average improvement on GPT-5.5 across six benchmarks. The optimized skill transfers across model scales and execution harnesses.

SkillGym is complementary: SkillOpt makes better skills, SkillGym uses those skills to make better models. An optimized skill produces higher-quality training data because its instructions are sharper and its evaluation criteria are more precise.

EvoSkill (anonymous, 2026): Evolutionary approach to skill generation through self-revision. Less controlled than SkillOpt.

Trace2Skill (anonymous, 2026): Extracts skills from agent execution traces. Produces skills from observed behavior rather than hand-authoring.

7.2. Skill and Tool Learning

Toolformer (Schick et al., 2023), Gorilla (Chen et al., 2023), and ToolLLM (Qin et al., 2023) teach models to call external APIs. Skills encode broader procedural knowledge beyond single API calls.

7.3. Self-Improvement and Self-Play

SPIN (Chen et al., 2024) and Self-Rewarding Language Models (Yuan et al., 2024) use model outputs as training signal. SkillGym differs by using skill-generated evals and controlled comparisons rather than self-evaluation.

7.4. Constitutional AI and RLAIF

Constitutional AI (Bai et al., 2022) uses AI-generated feedback. SkillGym uses evals derived from skill structure, not from a judge model. The verification is grounded in task requirements, not model preferences.

7.5. Code Generation Benchmarks

HumanEval (Chen et al., 2021) and SWE-bench (Jimenez et al., 2024) verify code execution. SkillGym extends verification to any domain with procedural structure.

7.6. Comparison with Our Results

SkillOpt achieves +23.5 points on GPT-5.5 using optimized skills. Our SFT experiments achieve +5.7 points on Qwen2.5-1.5B using raw skills. The magnitude difference is expected: SkillOpt uses a stronger model, optimized skills, and more benchmarks. Our contribution is demonstrating that the framework works at all on a 1.5B model with hand-authored skills and 30 training examples. The question is not whether SkillGym matches SkillOpt's performance, but whether the method scales when more skills and training examples are available.

8. Future Work

8.1. Validated

The following claims from the original framework are now supported by experimental evidence:

The comparative method produces measurable training signal. SFT fine-tuning on skill-assisted outputs improves performance on 67% of evaluation tasks with an 18.3% average composite improvement.

Small models can acquire domain competence. Qwen2.5-1.5B, a 1.5B parameter model, demonstrates measurable improvement in marketing tasks after skill-derived training.

SFT works at small scale. 30 training examples across 3 skills produce meaningful gains, particularly for tasks with objective verification criteria.

The skill is the verifier. Verification criteria embedded in SKILL.md files provide sufficient signal to score model outputs without building separate evaluation infrastructure.

8.2. Open Questions

The following questions remain for future experiments:

Scaling to more skills. Does the framework maintain its effectiveness when trained on 20+ skills with 500+ evaluation tasks? The cold-email results suggest that skills with objective criteria will continue to show strong improvement, while subjective skills may require more examples or better scoring functions.

DPO at scale. Can DPO outperform SFT when given hundreds of preference pairs? Our experiments with 17 pairs showed marginal improvement, but the framework predicts DPO becomes competitive at larger scales. This requires generating 100+ pairs with clear binary pass/fail criteria.

GRPO validation. Does group relative policy optimization produce cleaner signal than DPO for tasks with partial credit? This requires generating multiple attempts per task and scoring on a continuous scale.

SkillOpt integration. Do SkillOpt-optimized skills produce higher-quality training data? This requires running the SkillGym pipeline on both raw and optimized versions of the same skills and comparing delta distributions.

Cross-domain generalization. Does the framework work for code generation, schema operations, or deployment tasks? These domains have more objective verification criteria and may show stronger improvement than marketing.

Minimum delta threshold. What is the minimum skill delta for useful training signal? Tasks with small deltas (skill helps but not much) may add noise rather than signal to the training dataset.

Inter-annotator agreement. How consistent are human judgments of output quality across different evaluators? If automated scoring diverges from human judgment, the framework's training signal may be less reliable than it appears.

9. Conclusion

The agent skills ecosystem has accumulated something valuable: procedural policies with evaluable structure. SkillGym proposes a simple method to extract training signal from them: generate evals, run controlled comparisons, use the delta as reward.

Our experiments validate the core claim. Qwen2.5-1.5B, fine-tuned on just 30 examples from three marketing skills, achieves an 18.3% improvement in composite task performance. Cold-email tasks show 100% improvement rate. The model learns voice and criteria adherence most readily. SFT outperforms DPO at this scale, consistent with the framework's data requirements.

The curriculum is already written by thousands of independent contributors. The verification comes from the skill's own structure. The comparison isolates the skill's contribution. The resulting data is

grounded in actual task completion.

Two paths forward exist for skills: optimize the skill (SkillOpt) or use the skill to optimize the model (SkillGym). These are not competing approaches. They are complementary stages in a pipeline where better skills produce better training data, which produces better models, which execute skills more competently.

The economics favor this approach. Data generation from 30 tasks took approximately 2 hours of sub-agent execution. SFT training completed in 22 minutes on a single RTX 4050. The total cost is dominated by compute, not by data curation or human annotation.

If this scales, the implications are significant: the open-source ecosystem has already produced a vast curriculum of procedural knowledge. Converting this knowledge into training data would create a new class of specialized models that are small, efficient, and domain-competent. The path forward is empirical: run the pipeline on more skills, measure the results, and iterate.

References

1. Anthropic. "Skill Creator: A Skill for Creating and Iterating on Agent Skills." GitHub, 2026.

2. Bai, Y. et al. "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073, 2022.

3. Chen, M. et al. "Evaluating Large Language Models Trained on Code." arXiv:2107.03374, 2021.

4. Chen, X. et al. "Gorilla: Large Language Model Connected with Massive APIs." arXiv:2305.15334, 2023.

5. Chen, Z. et al. "SPIN: Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models." ICML,
2024.

6. DeepSeek-AI. "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning." arXiv,
2025.

7. Jimenez, C. E. et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" ICLR, 2024.

8. Li, Y. et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." NeurIPS, 2023.

9. Qin, Y. et al. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs." arXiv:2307.16789,
2023.

10. Shao, Z. et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." arXiv,
2024.

11. Schick, T. et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." arXiv:2302.04761, 2023.

12. Yang, Y. et al. "SkillOpt: Executive Strategy for Self-Evolving Agent Skills." arXiv:2605.23904, 2026.

13. Yuan, W. et al. "Self-Rewarding Language Models." arXiv:2401.10020, 2024.